Eightfold — Candidate Profile Aggregation Engine

Technical design, Step 1 — pipeline plan, canonical schema, merge/confidence policy, runtime config handling, and edge cases. No code; see the repo for the implementation.

---

1. Pipeline

Extract (per source, type-specific reader: CSV column map, JSON key map, PDF regex/heuristics) → **Normalize** (phone/date/skill/text, per-field functions, unparseable → null, never invented) → **Group evidence by field** → **Merge & resolve conflicts** (per field, single- vs. multi-valued policy) → **Score confidence** (deterministic formula, no model calls) → **Assemble canonical profile** → **Project** (runtime config reshapes output) → **Validate** (schema-check before returning). Every source is wrapped so one bad/missing source degrades to a warning, not a crash.

2. Canonical schema & normalized formats

Each candidate is one record: `candidate_id, full_name, emails[], phones[], current_company, current_title, skills[], overall_confidence, _meta`. Internally, every scalar field is `{value, confidence, sources, conflict, conflicting_values}` — not a bare value — so provenance survives until projection time decides whether to expose it. Multi-valued fields (`emails, phones, skills`) are lists of the same shape, one entry per distinct value. `_meta` records which sources were used, which failed (with why), and which raw values were dropped for failing normalization.

- **Phone** → E.164 (`+<cc><digits>`); bare 10-digit numbers assume one configurable default region (India, matching sample data); already-prefixed numbers pass through. Unparseable → null.
- **Dates** → `YYYY-MM` (month granularity matches the resume's own granularity); “Present/Current” → sentinel `"present"`, not a date.
- **Skills** → canonical display name via an explicit synonym table (`reactjs/react.js/react → React`); unrecognized terms are cleaned up (trim/case) and kept, not dropped — we don't get a canonicalization confidence bonus for them, but we never discard a stated skill.
- **Email** → lowercase + trim. **Text fields** (name, company, title) → whitespace-collapsed, casing preserved from source.

3. Merge / conflict-resolution policy & confidence

Single-valued fields (name, company, title): group evidence by a normalized comparison key. All sources agree → high confidence, no conflict. Disagreement: for `full_name` specifically, if one value is a token-subset of another (“Tejaswi Bhavani” ⊂ “Tejaswi Bhavani Hari”) we treat it as a refinement and keep the fuller one. Otherwise, a strict majority/plurality wins (flagged as a soft conflict, minority kept for audit); a true tie (every source disagrees, no majority) resolves to **null, confidence 0.0** — we never guess a winner out of a genuine contradiction.

Multi-valued fields (emails, phones, skills): union of distinct normalized values, each scored independently by how many sources corroborate it — not a single winner-take-all pick. Ordering (for paths like `emails[0]`) is confidence desc, then source-priority order (CLI argument order), then alphabetical, so it's fully deterministic.

Confidence formula: source-type base weight (structured > semi-structured > unstructured) + a bonus per corroborating source, capped at 0.98; a 0.85× haircut when a value was picked despite disagreement; 0.0 for an unresolved conflict or no evidence. Same inputs → same number, always.

4. Runtime custom-output config

The projection layer (`project.py`) never touches merge/confidence logic — it only reads three parallel flattened views of the canonical profile (values / confidences / sources), addressable by the same path syntax the config uses (`full_name, emails[0], skills[].name`). Per requested field: resolve `from` (or `path` if no remap), apply a `normalize` hint if given (mostly confirmations since canonical values are pre-normalized; `national` is the one real re-format), then apply `on_missing` (null/omit/error) if nothing resolved. `include_confidence` adds a `<field>_confidence/_sources` sibling per field, globally. Output is then validated against either the fixed default schema or a schema built dynamically from the config's own `type`/`required` declarations — so a new config never needs an engine code change, only a new config file.

5. Edge cases & deliberate scope cuts

- **Hard conflict, no majority** (e.g. three sources, three different companies) → null + confidence 0, all raw values kept in `conflicting_values` for audit. Never invented.
- **Corrupt / missing / malformed source** (unreadable PDF, missing file, header-less CSV) → caught per-source, run continues on whatever's left, source listed in `_meta.sources_failed` and surfaced as a CLI warning.
- **Same skill, different spelling** across sources (React/ReactJS) → canonicalized into one entry; this also means corroboration counts correctly even when sources never used the exact same string.
- **Config asks for a field that doesn't exist anywhere** in the canonical data → handled purely by `on_missing`, never fabricated.
- **Multiple distinct emails/phones** across sources, only some overlapping → unioned with per-value confidence, not forced into one arbitrary “truth.”

Left out under time pressure: full multi-job work-history/timeline reconciliation (only current role tracked); cross-candidate dedup/entity-resolution across a batch (one input set = one candidate per run); a polished UI (CLI only, per the brief's stated priority); the `phonenumbers`/`jsonschema` libraries (hand-rolled, swap-in compatible equivalents used instead, due to a network-restricted build environment).